

Video 3: Testing and Evaluating

How to debug with AI and maintain human oversight

Learning Objectives

By the end of this session, you should be able to:

- Explain the AI-assisted testing workflow and its role in development.
- Describe how unit tests are generated and used alongside AI-generated code.
- Outline the debugging loop for identifying and fixing failures.
- Identify key principles of code review and human oversight.
- Recognize common prompting failure modes and their impact on code quality.

Agenda

In this session, we'll discuss:

- AI-Assisted Testing Workflow
- Immediate AI-Based Test Generation After Coding
- Code Review and Human Oversight Discipline
- Failure Case Walkthroughs: Bad vs. Good Prompts
- “If AI Wrote It, You Own It” Principle

Testing and Debugging With AI

AI Excels at generating tests—but it cannot
Guarantee Correctness

AI-Assisted Testing Workflow: 4 Steps



This workflow turns the testing cycle into a collaborative loop: AI generates → you run → failures get fed back → AI corrects.

Important caveat: AI cannot guarantee correctness. It might generate tests that pass but still miss real bugs. Your job is to think about what it might have missed.

What to Ask For—and What Good Tests Cover

Prompt: "Write unit tests for `calculate_tax(income)` using Python `unittest`. Cover: normal incomes, boundary incomes (at bracket edges), zero income, and negative income."

Normal Cases

- Income of £15,000: Standard earner
- Income of £30,000: Basic rate band
- Income of £60,000: Higher rate
- Verify exact tax amounts against manual calculation

Edge Cases

- Exactly at the personal allowance: £12,570
- Exactly at the basic/higher rate boundary: £50,270
- Income of £0: Should return tax of £0
- Verify bracket boundaries don't cause off-by-one errors

Failure Cases

- Negative income: Should raise `ValueError`
- String input "fifty thousand": Should raise `TypeError`
- None input: Should raise an appropriate error
- Verify error messages are clear and informative

Debugging Loop: Turning Failures into Fixes

When a Test Fails: What to Do

- Step 1: Read the error message carefully—before anything else.
- Step 2: Try to understand what it is saying.
- Step 3: Use template 4—paste both code AND full error.
- Step 4: Ask AI to explain the root cause, not just give a fix.
- Step 5: Apply the fix and re-run all tests—not just the failing one.
- Step 6: If still failing, paste the new error and repeat.

What "Explain the Root Cause" Gets You

- You learn why the code failed—not just that it failed.
- You build genuine understanding of the code over time.
- You can catch the same class of error in future code.
- You stay in control rather than blindly applying AI patches.
- You own the code, which means you need to understand it.

Never apply an AI fix without understanding what it changed. An unexplained fix to one bug often creates two new ones.

Code Review and Human Oversight

"If the AI wrote it, you own it"—the most important principle in this course

5-Point Code Review Checklist for AI-Generated Code

01

**Does It Do What I
Asked?**

Run it with 3 different inputs you can verify manually. Check the output against your own mental calculation. Don't trust—test.

02

**Are There Hidden
Assumptions?**

AI often makes decisions without telling you. Look for hard-coded values, assumed data types, or logic paths you didn't specify.

03

**Is Anything Hard-
Coded That Should Be
a Variable?**

Tax rates, thresholds, file paths, API endpoints—any of these hard-coded in your logic is a future bug waiting to surface.

04

**Does It Handle Bad
Inputs?**

What happens with negative numbers? Empty strings? None values? If the AI didn't add validation, you need to ask for it explicitly.

05

**Is Any Part of It
Security-Sensitive?**

File access, user input, network calls, authentication—anything touching these must be manually reviewed; never trust it to AI output alone.

Run this checklist on every function the AI generates. It takes 2 minutes per function and saves hours of debugging later.

"If the AI Wrote It, You Own It"—What This Actually Means

When AI-generated code goes into a project—a tax calculator, an expenses tracker, anything—you are responsible for its correctness, security, and behavior. Not the AI.

What Ownership Means

- Reading every line before it goes into your project.
- Being able to explain what each function does.
- Testing every edge case yourself.
- Fixing bugs—with or without AI help.

What Ownership Does NOT Mean

- Writing every line yourself from scratch.
- Understanding every algorithm at a deep level before use.
- Never use AI to help you fix things.
- Avoiding AI tools out of fear or guilt.

How to Build Ownership Habits

- Never copy-paste without reading first.
- Ask AI to explain code in plain language.
- Add your own comments after AI generates code.
- Run your own tests independently of AI-generated tests.

The goal is not just to produce code quickly—it is to maintain long-term quality and control over what you build.

Failure Case Walkthroughs

What goes wrong—and why—is as important as
knowing what works

Failure Case 1: The Vague Prompt

The Vague Prompt

"Build a tax calculator."

What AI Returns

- Generic flat-rate tax function: No brackets.
- No error handling: Crashes on bad input.
- No structure: Just a loose function in a file.
- No comments: Impossible to verify.

The Better Prompt

"Build a Python function `calculate_tax(income)` that computes UK income tax for 2024/25. Use the following brackets: [list]. Return a dict with gross income, tax owed, and net income. Raise `ValueError` if income is negative."

What the Better Prompt Gets You

- Correct bracket structure matching real UK tax law.
- Input validation with meaningful error messages.
- Structured output you can test and verify.
- Code that matches your specification exactly.

Failure Case 2: Code That Runs but Is Wrong

The most dangerous failure mode: code that produces no errors but gives subtly incorrect results, and you didn't check.

The Scenario

- AI generates `calculate_tax()`: It runs without errors.
- For income of £30,000 it returns tax of £3,486.
- The correct answer is £3,486—so it passes your eye test.
- But for income of £50,270 (bracket boundary) it gives £7,540—wrong by £200.
- Off-by-one bracket logic—invisible until you test the boundary.

How to Catch Silent Bugs

- Always test at bracket boundaries—not just round numbers.
- Verify at least 3 values manually against an authoritative source.
- Ask AI to explain how it handles boundary conditions.
- Ask AI to add explicit comments at every boundary check.
- Compare against a known correct reference (HMRC calculator).

A test that passes on round numbers is not the same as a test that is correct. Silent bugs are the most expensive kind.

Failure Case 3: Accepting AI Output Without Reading It

The pattern: Generate → copy → paste → run → "it works" → move on. Without reading a single line.

What Gets Missed

- Assumptions the AI made silently.
- Hard-coded values that will break in production.
- Security issues in input handling or file access.
- Stubs or TODOs the AI left for you to complete.
- Logic that contradicts what you actually need.

The Cost

- Bugs discovered much later—when they're expensive.
- Code you don't understand and can't maintain.
- Technical debt that compounds over time.
- False confidence in code that has never been verified.
- Loss of control over your own project.

The Fix: Read First

- Make reading the code a non-negotiable step.
- Add your own comments as you read; this proves understanding.
- Ask AI: "What assumptions did you make in this code?"
- Ask AI: "What would make this code break?"
- Never ship code you could not explain to a colleague.

Reading AI code is not optional overhead—it is the core skill that separates effective vibe coders from frustrated ones.

Prompt Quality: A Direct Comparison

Small changes in prompt structure lead to significantly different outcomes. Here is the same task—two ways.

Weak Prompt

"Fix my expenses tracker."

- No context about what is broken.
- No error message provided.
- No code provided.
- Result: AI guesses, makes changes that may break other things, and returns code you cannot verify.

Strong Prompt

"Here is my add_expense() function: [CODE]. When I call it with a negative amount it does not raise an error. Here is the test that fails: [TEST + ERROR]. Explain what is wrong and provide a corrected version with the validation added."

- Result: AI targets the exact problem, explains the root cause, and returns a minimal, correct fix.

Key Takeaways

01

Test Immediately, Always

Generate tests right after generating code—not later. A function without tests is an assumption, not a feature.

02

You Own What AI Writes

Read it. Understand it. Add comments. Test it yourself. The AI authored it; you are responsible for it.

03

Failure Cases Teach the Most

The vague prompt, the silent bug, the unread output—learn to recognize these patterns before they cost you hours.

Next up: Video 4—Advanced AI-Assisted Development: Context windows, stubs, deduplication, and validation hooks

Summary

Here's a brief recap:

- The AI-assisted testing workflow defines how code generation, testing, and feedback are structured.
- Test generation establishes coverage across normal, edge, and failure scenarios.
- The debugging loop provides a method to identify root causes and correct errors.
- Code review and oversight define responsibilities for validating AI-generated code.
- Failure cases highlight common prompting issues and their impact on outcomes.

Learning Outcomes

You should now be able to:

- Generate and run unit tests immediately after writing code.
- Execute the debugging loop to identify and resolve failures systematically.
- Apply a structured code review checklist to evaluate AI-generated code.
- Diagnose prompting issues such as vague instructions, silent bugs, and unverified outputs.
- Maintain ownership and accountability for all AI-generated code in a project.